OPTIMIZACIÓN LOGÍSTICA DE TRANSPORTE PARA LA EMPRESA ÁZAMON

Práctica de Búsqueda Local



Pablo Abella Escudero, Mariona Casasnovas Simon y Núria López Encinas

Grau en Intel·ligència Artificial - UPC

Índice

1. Introducción
2. Implementación de estado
3. Creación de estados iniciales
   3.1 Estado inicial por días
   3.2 Estado inicial con componente aleatoria
4. Operadores
   4.1 Operador *MovePaquete*
   4.2 Operador *SwapPaquetes*
   4.3 Operador *SwapOferta*
5. Funciones heurísticas
   5.1 Heurístico básico
   5.2 Heurístico ponderado
   5.3 Heurístico con entropía
6. Experimentos
   6.1 Elección de los operadores
   6.2 Elección del estado inicial
   6.3 Ajuste de los parámetros de Simulated Annealing
   6.4 Evolución del tiempo de ejecución para hallar la solución en función del número paquetes y la proporción de peso transportable.
   6.5 Análisis del comportamiento del coste en función del número de ofertas
   6.6 Estudio de la felicidad a través de la ponderación de la función heurística
   6.7 Comparación con Simulated Annealing
   6.8 Variación del precio fijo de almacenamiento
7. Conclusiones

1. Introducción

En un contexto donde la demanda de servicios de distribución eficiente y oportuna es cada vez mayor, la optimización logística se presenta como un desafío esencial para las empresas. Las compañías dedicadas al comercio y distribución en línea, como la ficticia Ázamon, necesitan gestionar de manera eficaz sus envíos, minimizando tanto los costes de transporte como los de almacenamiento, a fin de mejorar su competitividad. Esta práctica se enfoca en la aplicación de algoritmos de búsqueda local para resolver el problema logístico de asignación de paquetes a empresas de paquetería, teniendo en cuenta la variabilidad de las ofertas diarias de cada proveedor en términos de peso máximo, precio por kilogramo y tiempos de entrega.

Ázamon debe procesar diariamente una gran cantidad de paquetes, cada uno con características específicas de peso y prioridad de envío, establecidas por los clientes. Estas prioridades, que abarcan desde el día siguiente hasta un máximo de cinco días, implican diferentes costes y tiempos de espera, lo que agrega una dimensión de complejidad a la planificación de envíos. Además, la empresa enfrenta desafíos adicionales, como el coste de almacenamiento por kilogramo y día y la necesidad de optimizar la experiencia del cliente mediante el envío anticipado de paquetes, lo cual se traduce en una mayor satisfacción y aumento de la “felicidad”.

En esta práctica, se busca desarrollar una asignación eficiente de paquetes a las ofertas de las empresas de transporte considerando tres factores clave: los costes de transporte, los costes de almacenamiento y la “felicidad” del cliente derivada del adelanto en la entrega.

2. Implementación del estado

En nuestro caso, la elección de la representación del estado fue guiada por la necesidad de capturar y evaluar los elementos clave del problema de forma eficiente, de modo que fuera posible aplicar los operadores de búsqueda local y calcular el coste y la felicidad de cada configuración.

Tras analizar los componentes del problema, decidimos usar la clase Estado para contener varios atributos que representan tanto los datos de entrada como los resultados intermedios del proceso de búsqueda:

Asignaciones: Optamos por usar un diccionario para almacenar las asignaciones de paquetes a ofertas. Este enfoque es flexible y permite representar de manera clara cómo se distribuyen los paquetes entre las ofertas.

Coste y Felicidad: Elegimos almacenar el coste y la felicidad como parámetros para no tener que calcularlos en cada nuevo estado y poder simplemente añadir o restar los cambios después de aplicar cada acción. El coste total refleja la eficiencia de la solución, mientras que la felicidad total refleja el bienestar asociado con cada configuración. Al tener estos valores como atributos separados, podemos evaluarlos y compararlos fácilmente entre diferentes estados.

Transporte y Paquetes: Las instancias de las clases *Transporte* y *Paquetes* se incluyen para, de esta forma, poder acceder a las diversas instancias de las clases *Oferta* y *Paquete*, los datos que en estas se almacenan y, además, para acceder a los métodos que hemos implementado tanto en *Transporte* como en *Paquetes* para operar con estos datos.

Generación de acciones: Primeramente calculamos el peso ocupado y el peso libre o restante de cada una de las ofertas que utilizamos. Estos dos datos los usaremos para poder determinar qué ofertas tienen espacio libre y cuáles no, así como la cantidad de este. Como lo que buscamos con las acciones es mover los paquetes entre las ofertas para así poder mejorar tanto el coste como la felicidad, es necesario saber si estos movimientos se pueden llevar a cabo o si, por el contrario, superan la capacidad de alguna oferta. Para evitarnos tener que estar constantemente calculándolos, pues son necesarios para determinar cada una de las acciones, realizamos las operaciones pertinentes únicamente una vez al inicio, y guardamos estos valores en listas. La generación de las acciones propiamente dicha empieza con un bucle que recorre todas las ofertas usadas que tenemos guardadas en el diccionario. Se hace de esta forma porque de cada una de ellas es necesario comprobar: si todos sus paquetes se pueden mover en bloque a alguna otra oferta (acción *Swap Oferta*) y, si para cada paquete este se puede mover a alguna otra oferta (acción *Move Paquete*) o se puede intercambiar con algún paquete de alguna otra oferta (acción *Swap Paquete*).

La representación elegida nos permite manejar eficientemente tanto las asignaciones como los cálculos asociados al coste y la felicidad, y modificar los estados durante el proceso de búsqueda local. Además, al encapsular todos los elementos relevantes en una única clase, la representación del estado facilita la implementación de los operadores de búsqueda y permite explorar de manera efectiva el espacio de soluciones.

3. Creación de los estados iniciales

En la optimización mediante algoritmos de búsqueda local, la selección de un estado inicial adecuado puede influir significativamente en la eficiencia y efectividad del proceso de búsqueda. En esta sección se presentan y analizan dos posibles estados iniciales, conforme a los requisitos establecidos

en la práctica. La elección de estos dos estados responde a la necesidad de explorar diferentes enfoques iniciales que puedan influir en el rendimiento del algoritmo y, en última instancia, en la calidad de la solución final obtenida.

3.1. Estado inicial por días

El primer estado inicial con el que contamos es el que nos pareció más intuitivo teniendo en cuenta que para usar algoritmos de búsqueda local se necesita empezar la búsqueda desde un estado solución del problema para poder recorrer el espacio de soluciones. Por ello se cumple que todos los paquetes son asignados a una oferta de transporte consiguiendo que los paquetes lleguen dentro del plazo indicado por su prioridad y el peso de los paquetes asignados a una oferta no superará su capacidad

La función encargada de la generación es *estado_inicial* la cual genera el estado inicial del problema teniendo clasificados los paquetes por prioridad y las ofertas por día. Crea un estado inicial que cumple todas las prioridades y tiene felicidad=0.

La solución que obtenemos es sencilla y podría considerarse que tiene un nivel de compactación medio, los paquetes quedaràn suficientemente distribuidos entre las ofertas disponibles aunque haya posibilidad de que lo estén todavía más. Es una solución de partida que se prevé cercana a la solución que deberá encontrar el algoritmo y que con los operadores convenientes podrà llegar ella. El coste de la creación de esta solución inicial O(n), donde n es el número de paquetes.

3.2. Estado inicial con componente aleatoria

Para el segundo estado inicial queríamos cambiar lo poco variable e igual de distribuïda que iba a estar la primera solución inicial. Para ello, decidimos seguir respetando las restricciones que debía cumplir el estado inicial para que este pudiera ser solución, pero además introducir algún elemento que nos permitiera partir de una solución suficientemente distante a la ya creada y que ademàs incorporara un componente que nos permitiera tener variabilidad en las posibles soluciones iniciales de un mismo contexto. Es por ello que decidimos incluir una componente que aportara aleatoriedad, pero sin dejar de cumplir las restricciones.

La función encargada de la generación es *estado_inicial_aleatorio* que genera el estado inicial del problema asignando a cada paquete una oferta aleatoria de su prioridad. Crea un estado inicial que cumple todas las prioridades y que tiene felicidad=0

Al generar la solución cada paquete es asignado aleatoriamente a una de las ofertas que cumple con su misma prioridad. De esta manera las soluciones que obtenemos con esta inicialización tienden a tener los paquetes más distribuidos entre las ofertas, no se encuentran tan compactados. Esto puede significar que se pueda llegar a realizar una exploración más amplia del espacio de soluciones.
El coste de la creación de esta solución inicial O(n+k), donde n es el número de paquetes y k es el número de ofertas.

4. Implementación de los operadores

Los operadores juegan un papel fundamental en el diseño de algoritmos de búsqueda local, estos determinan cómo se modifican los estados durante el proceso de exploración de soluciones. Para la realización de nuestra práctica hemos implementado tres operadores diferentes que nos han parecido los más adecuados para la exploración del espacio de soluciones.

4.1. Operador *MovePaquete*

Este operador es sin duda el operador más básico que se puede encontrar para el problema planteado en la práctica. Consiste en la acción de mover un paquete de una oferta a otra. Su uso siempre respeta la prioridad de la oferta, permite mover un paquete para que este se entregue antes (lo cual comportaría una mejora para la "felicidad") colocando en una oferta con fecha de entrega más cercana, pero nunca moverá un paquete a una oferta que lo entregue posteriormente a lo indicado por su prioridad. Al permitir únicamente movimientos que adelanten la fecha de entrega o mantengan el compromiso original, este operador ofrece una forma sencilla pero controlada de optimizar las asignaciones, enfocándose en maximizar la satisfacción sin añadir retrasos. El coste de ramificación de este operador sería O(n*k) siendo n el número de paquetes i k el número de ofertas.

4.2. Operador *SwapPaquetes*

El operador *SwapPaquetes* realiza el intercambio de paquetes entre dos ofertas diferentes, respetando las restricciones de prioridad de cada paquete. Al igual que el operador *MovePaquete*, *SwapPaquetes* garantiza que ningún paquete se asignará a una oferta que no cumpla con su tiempo de entrega deseado, permitiendo únicamente intercambios que anticipen o mantengan la fecha de entrega especificada por el cliente. Este operador resulta útil para ajustar las asignaciones de manera que se logre una distribución más equilibrada de los paquetes entre las ofertas, facilitando entregas anticipadas que incrementen la "felicidad" del cliente. Además, al permitir la permutación de

paquetes, este operador contribuye a una exploración más diversa del espacio de soluciones. El coste de ramificación de este operador sería O(n*(n-1)) = O(n^2)

4.3. Operador *SwapOferta*

El operador *SwapOferta* permite intercambiar todos los paquetes asignados de una oferta a otra en una única operación. A diferencia del operador *MovePaquete*, que realiza cambios individuales, *SwapOferta* agiliza el proceso de exploración en el espacio de soluciones al ejecutar de manera simultánea múltiples movimientos de paquetes desde una oferta inicial hacia otra. Este operador es particularmente útil cuando se identifican ofertas que pueden cumplir mejor con los requisitos de coste o tiempos de entrega para varios paquetes a la vez, optimizando así la asignación en un solo paso. Además, al reducir el número de movimientos unitarios necesarios para reubicar un conjunto de paquetes, *SwapOferta* mejora la eficiencia computacional del algoritmo y permite realizar ajustes estratégicos en menos iteraciones. El coste de ramificación de este operador sería O(k*(k-1))=O(k^2)

5. Funciones Heurísticas

Nuestra implementación de la práctica cuenta con 3 heurísticas diferentes.

5.1. Heurístico básico

El heurístico básico consiste en una expresión simple diseñada específicamente para las primeras etapas de experimentación, donde el objetivo principal es reducir el coste total de la solución. Este heurístico se implementa como $-$*self. coste*, de modo que el algoritmo de búsqueda pueda enfocarse únicamente en minimizar los costes asociados a las asignaciones, sin considerar otros factores como la "felicidad" del cliente. Al establecer esta función de evaluación sencilla, el heurístico básico permite evaluar rápidamente el impacto de los cambios en las asignaciones en términos de coste, lo cual facilita los ajustes iniciales y proporciona una base de comparación para futuros experimentos con heurísticos más complejos.

5.2. Heurístico ponderado

El heurístico ponderado incorpora los dos criterios que queremos optimizar. Por un lado poder maximizar la felicidad y por otro minimizar el coste. Para poder incorporar la ponderación de estos valores hemos añadido un parámetro k el cual nos permitirá realizar experimentos para analizar qué cambios notaremos cuando le demos un mayor peso a la felicidad o al coste La expresión que nos queda es la siguiente: *self.k*float(self._felicidad)-self._coste*. f.

5.3. Heurístico con entropia

El último heurístico que diseñamos incorpora el concepto de entropía para lograr una asignación más equilibrada y optimizada. A diferencia del heurístico básico, que se centra únicamente en minimizar el coste, este heurístico considera tres factores: minimizar el coste, reducir la entropía y maximizar la "felicidad" del cliente. La función heurística se define como *self.k * float(self._felicidad) - self._coste - self.e * total_entropy*, donde *self.k* y *self.e* son factores de ponderación que ajustan la importancia de la felicidad y la entropía en la evaluación de la solución.

Este enfoque permite que el heurístico no solo se enfoque en la reducción de costes, sino también en alcanzar una distribución más equilibrada de los paquetes entre las ofertas. Al aumentar el valor de "felicidad" mediante entregas anticipadas, este heurístico también promueve la satisfacción del cliente, proporcionando así una solución más completa y adaptada a múltiples objetivos.

Pese a que en un principio nos pareció buena idea poder contar con esta tercera heurística, a medida que hemos ido avanzando en la práctica nos hemos dado cuenta de que su uso no mejora el problema según lo que se plantea en el enunciado. Compactar los paquetes en un principio no influirá en el hecho de minimizar el coste y maximizar la felicidad, no tiene un impacto en la mejoría de la solución que se encuentre.

6. Experimentos

6.1. Elección de los operadores

El objetivo de este experimento es determinar qué conjunto de operadores produce los mejores resultados para una función heurística que optimice únicamente el coste, por ello usaremos el heurístico básico, en un escenario donde se deben enviar 50 paquetes y la proporción del peso transportable por las ofertas es 1:2. El algoritmo a usar es Hill Climbing. Como estado inicial decidimos usar el estado inicial que no contiene el componente aleatorio.

Como planteamiento para elegir qué operadores són los que producen mejores resultados hemos decidido estudiar el impacto que tiene el uso del operador *SwapOferta*. Hemos decidido centrarnos en este operador ya que *MovePaquete* y *SwapPaquete* los hemos considerado necesarios para poder asegurar que el algoritmo podrá recorrer o acceder a todo el espacio de soluciones. Por ello hemos querido ver si el uso de *SwapOferta* es significativo para poder encontrar soluciones mejores debido a

que este operador equivale a muchas operaciones de *MovePaquete*. Al evaluar su efectividad, podremos determinar si su inclusión mejora sustancialmente los resultados o si, por el contrario, su uso es redundante en comparación con los otros operadores.

Nos planteamos si *SwapOferta* aporta alguna mejora.

- Consigue que el algoritmo encuentre una solución mejor
- Consigue que el algoritmo encuentre la solución en un tiempo o nº de pasos menor

Observación	El uso del operador *SwapOferta* al conjunto de operadores supone una mejora para el programa
Planteamiento	Estudiamos la ejecución de diferentes problemas y comparamos cada uno de ellos con el uso del operador *SwapOferta*
Hipòtesis	Usar *SwapOferta* empeora o no mejora el rendimiento del programa (Ho), *SwapOferta* mejora el rendimiento del programa(H1)
Método	- Usar Hill Climbing - Crear 10 semillas de forma aleatoria - Con cada semilla realizar 10 ejecuciones para obtener mejores resultados - Comparar los tiempos de ejecución según el uso del operador o no - Comprobar la calidad de las soluciones obtenidas con el valor del heurístico

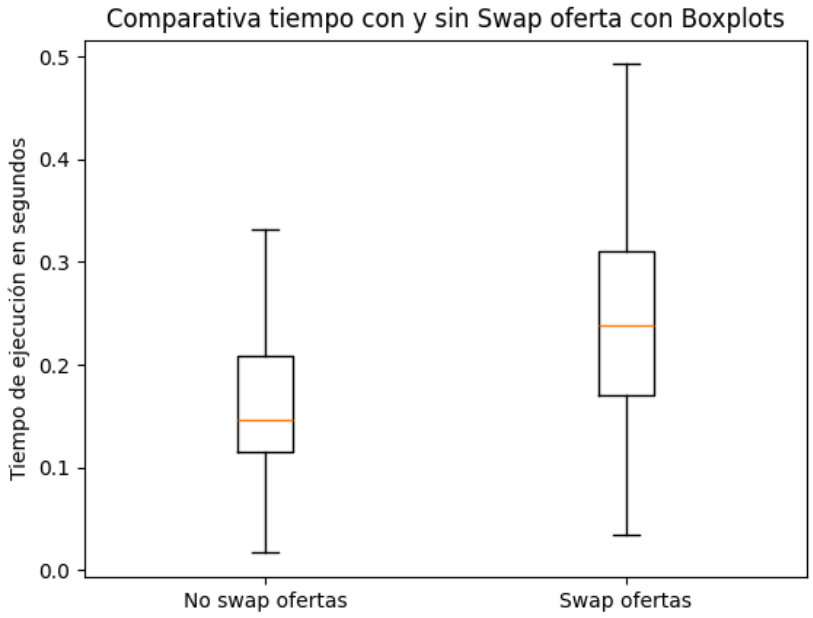
6.1.1. Resultados

Resultados Tiempo de ejecución(los valores en verde son los más bajos de cada fila)

Semillas	Usando *SwapOfertas*	No usando *SwapOfertas*
7729	0.2275	0.1260
9390	0.2480	0.2553
6345	0.2687	0.1712
6990	0.0568	0.0437
9671	0.1683	0.2214
1499	0.3261	0.1355
1681	0.4925	0.3316

8109	0.3239	0.1573
5748	0.1780	0.1121
9990	0.0349	0.0169
Media	0.23547	0.1620

Analizando los resultados, observamos que el tiempo de ejecución cuando se utiliza el operador *SwapOferta* es mayor en la gran mayoría de las veces. Aunque las unidades de tiempo están en segundos y el incremento no es excesivamente significativo, sí que se observa un aumento notable en comparación con la restricción de su uso. El aumento no parece ser lo suficientemente grande como para comprometer la eficiencia general del algoritmo en escenarios específicos.

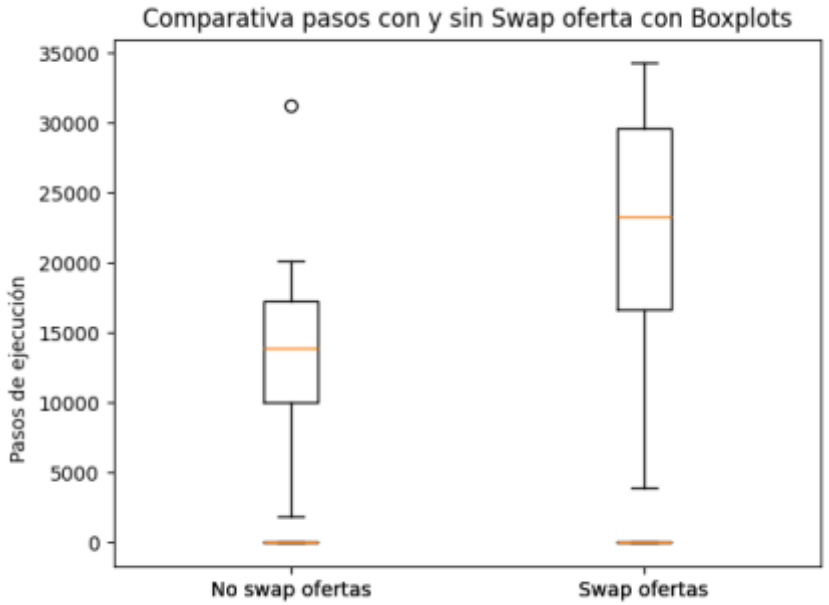


Resultados nº de pasos

Semillas	Usando *SwapOfertas*	No usando *SwapOfertas*
7729	24457.4	9495.4
9390	25342.9	17664.8
6345	31079.3	15966.5
6990	6300.4	4373.6
9671	17010.4	20152.7
1499	32268.5	12081.8
1681	34278.8	31298.4
8109	22203.5	15647.5

5748	16621.5	11764.5
9990	3899.5	1864.5
Media	19946.13	14230.93

El número de pasos realizados al utilizar un operador adicional es claramente mayor, como ya se podía predecir. Esto se debe a que el operador *SwapOferta*, al implicar la reubicación de todos los paquetes de una oferta a otra, incrementa la cantidad de operaciones que deben ejecutarse en cada iteración. Aunque este aumento en el número de pasos podría parecer significativo, su impacto debe analizarse en conjunto con los resultados globales, ya que puede haber situaciones donde este incremento sea compensado por una mejora en la calidad de la solución.

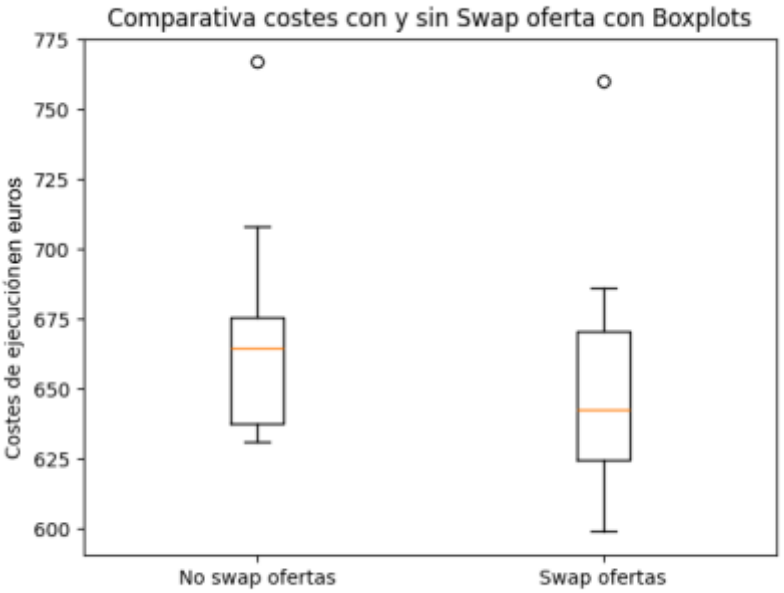


Resultados Coste

Semillas	Usando *SwapOfertas*	No usando *SwapOfertas*
7729	646.02	674.985
9390	760.04	766.870
6345	599.02	631.015
6990	632.40	632.745
9671	622.10	652.4975
1499	686.10	708.120
1681	610.89	631.123

8109	656.09	664.430
5748	639.69	665.360
9990	675.69	676.140
Media	646.804	664.31855

Los resultados del valor del coste, que es lo que determina la calidad de una solución, muestran que el uso del operador *SwapOferta* supone un avance hacia la obtención de mejores soluciones. Este avance no es constante, ya que en algunas ocasiones el progreso es más significativo que en otras. Sin embargo, en general, el uso de este operador contribuye positivamente a reducir el coste total, lo que indica que, *SwapOferta* puede ser una herramienta valiosa para encontrar mejorares soluciones al problema planteado.



6.1.2. Conclusiones

Dado que en esta práctica nos centramos en encontrar la mejor solución posible y el tiempo de ejecución pasa a un segundo plano, hemos considerado que el uso del operador *SwapOferta* es significativo para lograr nuestro objetivo. A pesar del incremento en el tiempo de ejecución, el impacto positivo en la calidad de la solución lo convierte en una herramienta valiosa, ya que permite avanzar de manera más eficiente hacia la optimización del coste, que es el criterio principal.

6.2. Elección del estado inicial

El objetivo de este experimento es determinar qué estrategia de generación de la solución inicial produce mejores resultados para la función heurística utilizada en el experimento anterior, utilizando

el algoritmo de Hill Climbing. A partir de los resultados obtenidos, se establecerá la estrategia de generación de la solución inicial que emplearemos.

Observación	Podemos obtener mejores soluciones en función de què solución inicial usemos
Planteamiento	Escogemos dos métodos de inicialización y observamos el coste de la solución final
Hipòtesis	La solución inicial que incorpora aleatoriedad encontrará mejores soluciones (Ho) El método de inicialización sin componente aleatoria es mejor (Ha)
Método	- Usar Hill Climbing - Crear 10 semillas de forma aleatoria - Con cada semilla realizar 10 ejecuciones para obtener mejores resultados - Comprobar la calidad de las soluciones obtenidas con el valor del heurítico

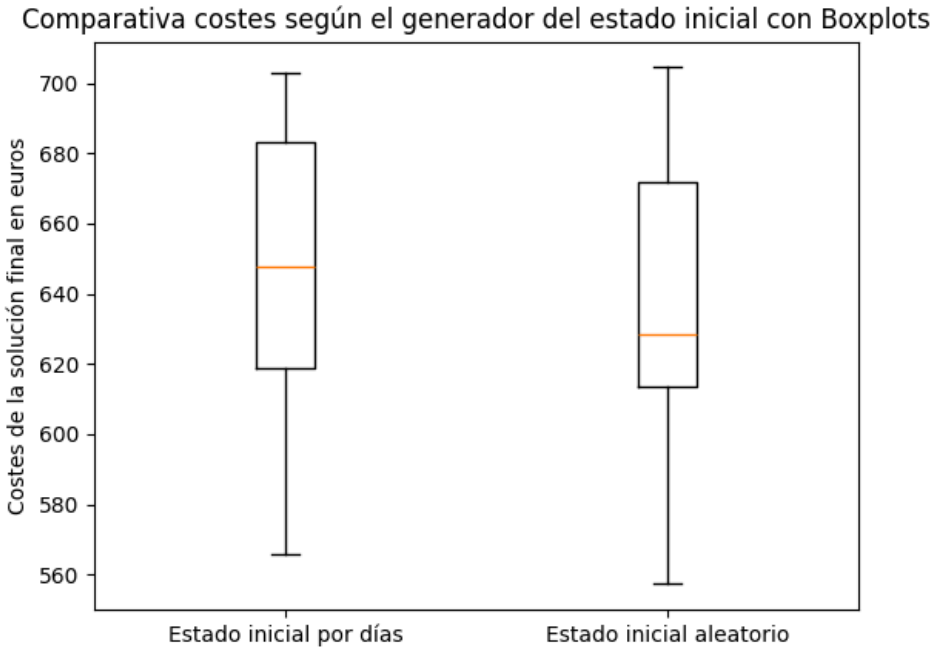
6.2.1. Resultados

Resultados Coste

Semillas	Estado inicial aleatorio	Estado Inicial por días
2198	684.125	689.430
6484	564.770	578.32
9785	704.430	702.990
1063	615.575	625.985
9050	557.325	565.830
2673	665.860	670.563
6587	618.280	639.0
8628	612.730	616.34
2580	638.763	655.980
5160	673.746	687.170
Media	738.160	744.576

Los resultados muestran que los costes de la solución inicial, al utilizar un estado inicial aleatorio, son notablemente mejores que cuando se emplea un estado inicial sin componente aleatorio. Esto indica que, al incorporar aleatoriedad en el proceso de generación de la solución, es posible alcanzar

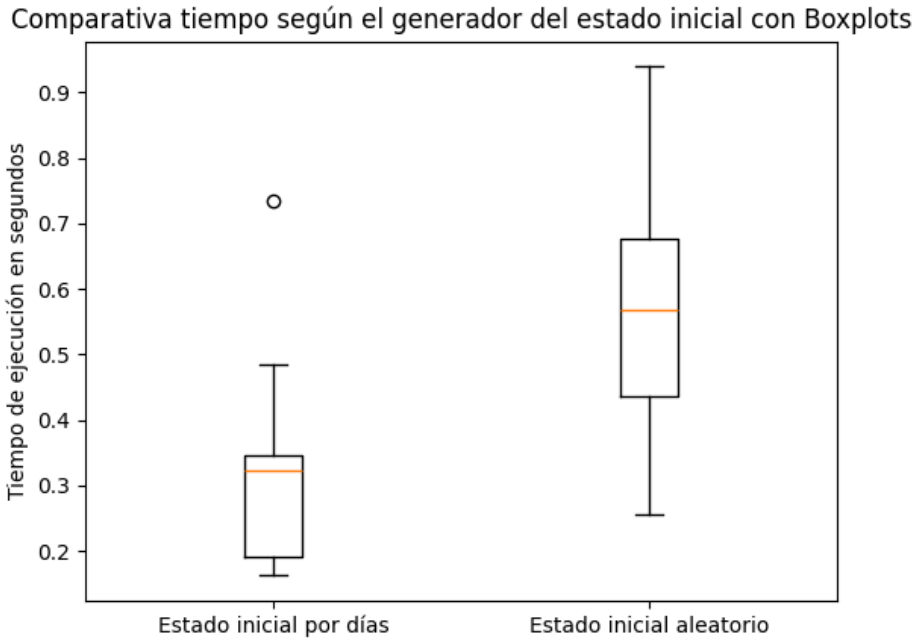
soluciones más favorables, superando los resultados obtenidos cuando la solución inicial se genera de manera determinista, sin variabilidad. Este comportamiento sugiere que la aleatoriedad en el estado inicial puede desempeñar un papel clave en la mejora de los resultados del algoritmo.



Resultados Tiempo de ejecución

Semillas	Estado Inicial por días	Estado inicial aleatorio
2198	0.7340	0.9393
6484	0.2146	0.2560
9785	0.3507	0.6159
1063	0.3284	0.4792
9050	0.1834	0.5217
2673	0.4835	0.6817
6587	0.3200	0.4217
8628	0.3241	0.7541
2580	0.1622	0.6627
5160	0.1690	0.3761
Media	0.33099	0.5687

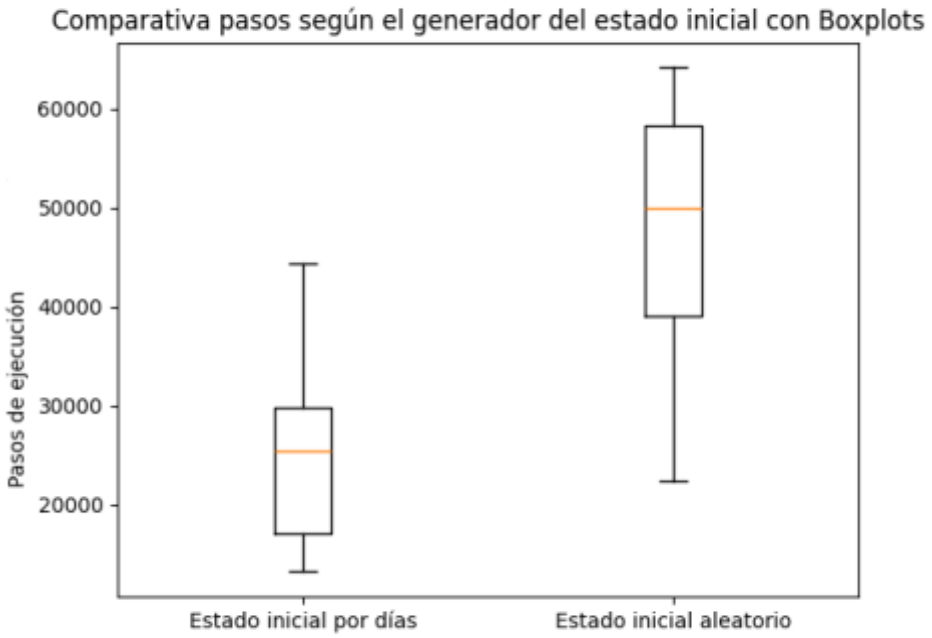
Los tiempos de ejecución son menores cuando se utiliza un estado inicial sin componente aleatoria, lo que significa que el algoritmo alcanza su solución en menos tiempo. Esto puede explicarse por el hecho de que, al no incorporar aleatoriedad en el estado inicial, el proceso de búsqueda sigue una trayectoria más directa y menos impredecible, lo que reduce la cantidad de pasos necesarios para llegar a una solución.



Resultados nº de pasos

Semillas	Estado Inicial por días	Estado inicial aleatorio
2198	39872.5	53638.5
6484	18788.0	22371.8
9785	29893.4	51250.3
1063	29821.0	44364.0
9050	16543.0	48674.0
2673	44421.9	63640.5
6587	26621.3	37382.5
8628	24362.8	64167.5
2580	13262.3	59818.2
5160	13853.4	34241.2
Media	26773.86	45954.75

El número de pasos realizados con el estado inicial sin componente aleatorio es considerablemente menor, lo que implica que el algoritmo recorre un espacio de soluciones más reducido. Este comportamiento está directamente relacionado con el tiempo de ejecución, ya que al realizar menos pasos, el algoritmo termina más rápido. La falta de aleatoriedad en el estado inicial permite una búsqueda más directa y eficiente, pero a costa de explorar menos opciones dentro del espacio de soluciones. Esto sugiere que, aunque el proceso es más rápido, también puede estar limitando la capacidad del algoritmo para encontrar soluciones más óptimas en comparación con el uso de un estado inicial aleatorio.



6.2.2. Conclusiones

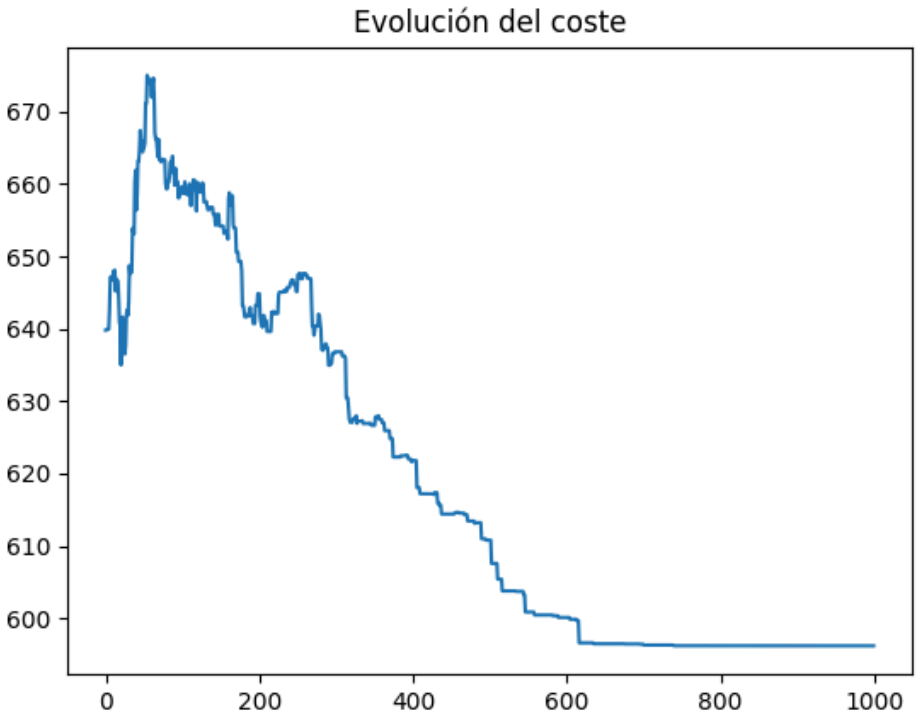
Los resultados obtenidos en este experimento podrían compararse con los alcanzados en el experimento anterior. Aunque el uso de un estado inicial sin componente aleatoria resulta en tiempos de ejecución más rápidos y un menor número de pasos, los resultados en términos de coste son significativamente mejores cuando se utiliza el estado inicial con componente aleatoria. La aleatoriedad en el estado inicial permite explorar un mayor espacio de soluciones, lo que a su vez puede llevar a encontrar soluciones más óptimas, aunque a costa de un mayor tiempo de ejecución.

6.3. Ajuste de los parámetros de Simulated Annealing

Para determinar qué parámetros son los adecuados hemos decidido probar 3 combinaciones diferentes y analizar el impacto de cada combinación sobre el problema y la calidad de la solución a la que se llega:

6.3.1. Un parámetro K bajo y lambda alta

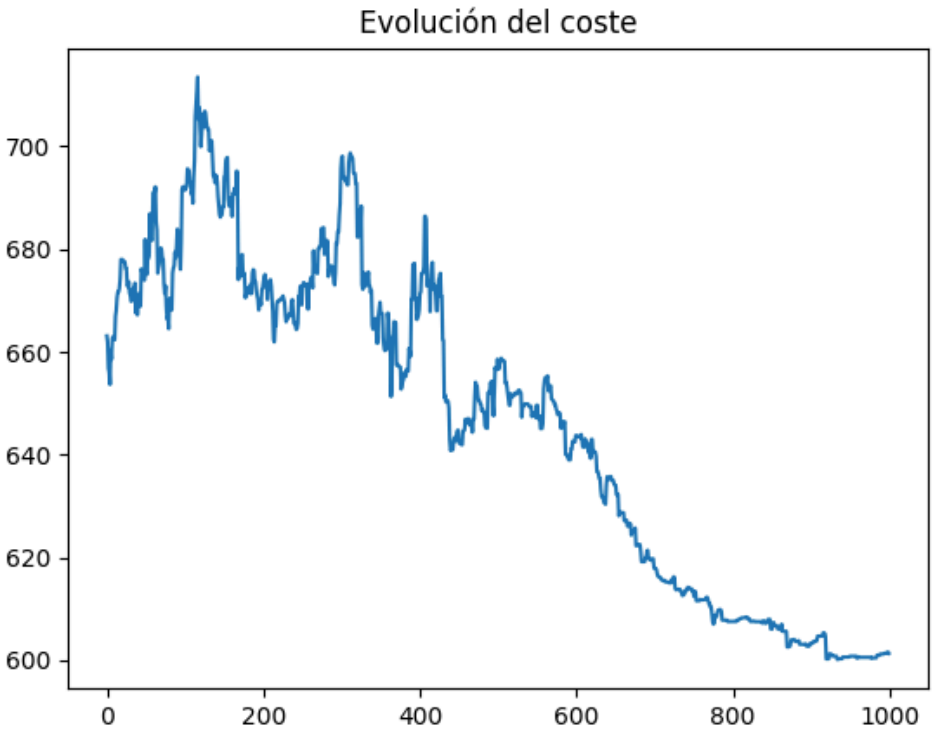
El valor de k controla la probabilidad de aceptar peores soluciones, hace que el algoritmo sea menos propenso a aceptar soluciones peores, incluso al principio, lo cual aumenta el determinismo y disminuye la exploración. El parámetro lambda controla cómo decrece la temperatura con cada iteración. Un lambda alto significa que la temperatura disminuye más lentamente, lo que permite que el algoritmo mantenga una exploración más prolongada del espacio de soluciones antes de "enfriarse" por completo.



k=10, lam=0.01 limit=1000

6.3.2. Un valor medio de K i lambda

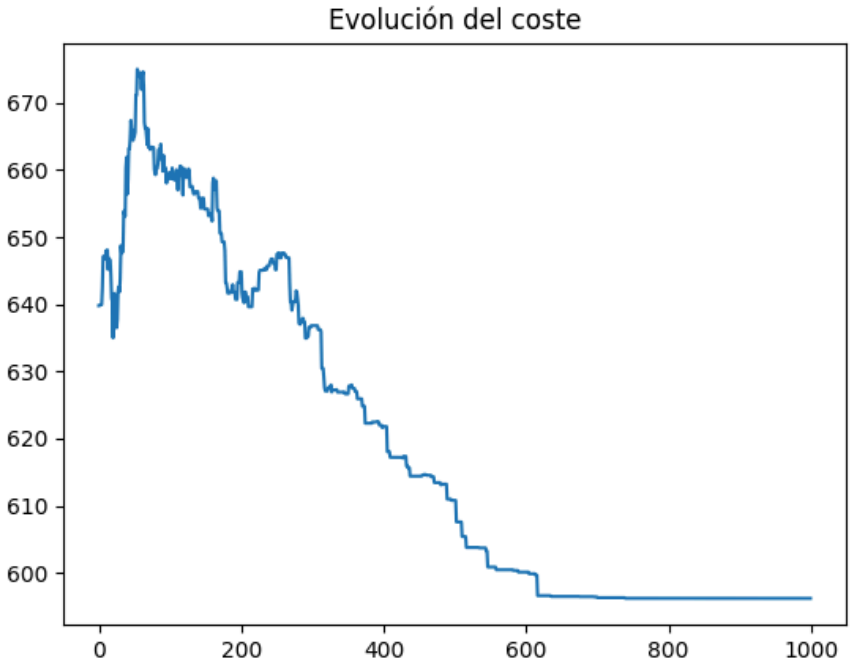
Se debería lograr un equilibrio entre exploración y explotación, lo que permite al algoritmo explorar el espacio de soluciones sin ser ni demasiado rígido ni demasiado permisivo. Un valor medio de K hará que el algoritmo permita aceptar soluciones peores en las primeras iteraciones, lo que ayuda a escapar de los óptimos locales y explorar un espacio de soluciones más amplio. Un valor medio de lambda hará que la temperatura disminuya de manera constante, permitiendo que el algoritmo tenga suficiente tiempo en la fase de exploración inicial para aceptar soluciones subóptimas, pero sin mantener la "temperatura" alta por demasiado tiempo.



k=50, lam=0.005, limit=1000

6.3.3. Un valor alto de K i un valor muy pequeño de lambda

Un valor alto de K implica que el algoritmo tiene una alta probabilidad de aceptar soluciones peores, incluso con una temperatura baja, significa que el algoritmo seguirá aceptando soluciones peores con frecuencia, manteniendo una exploración más prolongada y permitiendo que el algoritmo escape de óptimos locales incluso en etapas avanzadas. Un valor muy pequeño de lambda provocará que la temperatura se mantenga alta durante muchas iteraciones, prolongando la fase de exploración del algoritmo por mucho tiempo antes de entrar en un régimen de explotación más restringido.



k=100, lam=0.001, limit=1000

6.3.1. Conclusiones

La combinación de un parámetro K bajo y lambda alta nos ha parecido la más adecuada basándonos en los resultados obtenidos.

6.4. Evolución del tiempo de ejecución para hallar la solución en función del número paquetes y la proporción de peso transportable.

El objetivo de este experimento es analizar la tendencia en el tiempo de ejecución del programa en relación con los valores de la proporción del peso transportable y el número de paquetes. A partir del escenario de los apartados anteriores, se busca observar cómo varía el tiempo requerido para hallar la solución en función de estos dos parámetros. El experimento utilizará el algoritmo de Hill Climbing y la misma heurística previamente establecida. Se trata de estudiar la posible tendencia del tiempo de ejecución del programa en función de los valores de la proporción del peso transportable o del número de paquetes.

a) Proporción del peso transportable

Para analizar la tendencia del tiempo de ejecución en función de la proporción del peso transportable, fijaremos el número de paquetes en 50 y utilizaremos el estado inicial determinado en el experimento 2, junto con la función heurística que se centra únicamente en optimizar el coste de la solución.

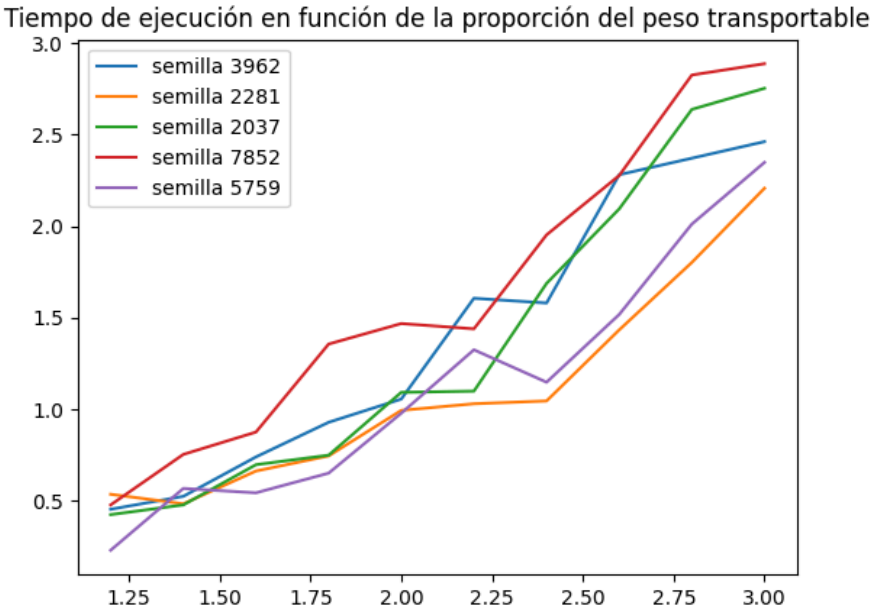
Con el fin de mejorar la fiabilidad de los resultados, usaremos un total de cinco semillas diferentes. Para cada una de estas semillas, y por cada valor de la proporción del peso transportable, realizaremos un total de 10 ejecuciones. Calcularemos la media de los tiempos obtenidos en estas ejecuciones para observar y representar gráficamente la tendencia general del tiempo de ejecución.

i) Método

- Creamos 5 semillas diferentes
- Para cada valor de la proporción del peso realizamos 10 ejecuciones y usamos su media

Esperamos que los resultados muestren un aumento en el tiempo de ejecución a medida que se incrementa la proporción de peso transportable. Esto se debe a que un mayor espacio disponible para colocar paquetes generará un espacio de soluciones más amplio, lo cual probablemente se traduzca en un aumento en el tiempo necesario para explorar y hallar las soluciones que buscamos.

ii) Resultados



Como anticipamos, la tendencia observada muestra un claro aumento en el tiempo de ejecución conforme incrementa la proporción de peso transportable. Esto confirma lo que habíamos predicho, el incremento en la proporción de peso aumenta el espacio de soluciones, lo que prolonga el tiempo necesario para alcanzar la solución que buscamos.

b) Número de paquetes

El método que usaremos será prácticamente idéntico al utilizado en el estudio de la proporción de peso transportable. En esta ocasión, aplicaremos el mismo procedimiento, pero variando el número de

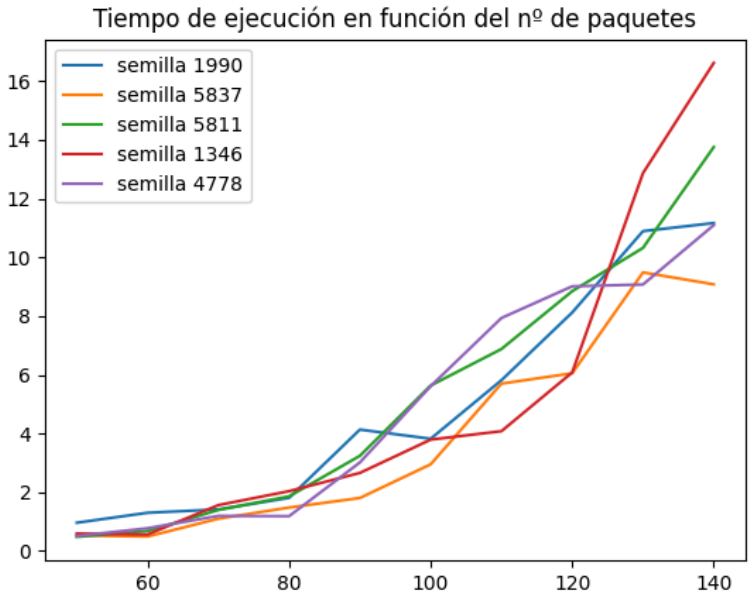
paquetes, que incrementaremos de 10 en 10 en cada iteración para observar su impacto en el tiempo de ejecución.

i) Método

- Creamos 5 semillas diferentes
- Para cada valor del número de paquetes realizamos 10 ejecuciones y usamos su media

Suponemos que el tiempo de ejecución a medida que aumenta el número de paquetes también será mayor ya que cada vez habrá más paquetes que pueden ser movidos,lo cual incrementa el número de posibles acciones y, en consecuencia, el espacio de soluciones a explorar.

ii) Resultados

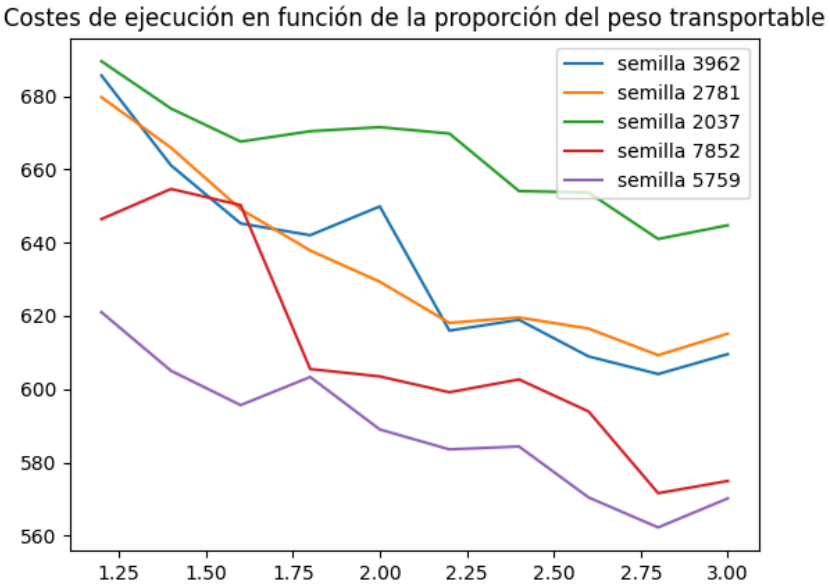


En el gráfico se puede observar un incremento exponencial, observamos que existe una clara tendencia al alza. Esto indica que incrementar el número de paquetes aumenta el tiempo de ejecución, como era de esperar debido al crecimiento del espacio de soluciones. El patrón sugiere que el número de paquetes influye directamente en la complejidad y el tiempo de procesamiento del algoritmo.

6.5. Análisis del comportamiento del coste en función del número de ofertas

En el apartado anterior observamos que el tiempo de ejecución del programa aumentaba al incrementar el valor de la proporción de las ofertas, lo cual sugiere que explorar una mayor cantidad de soluciones posibles requiere más tiempo. Ahora, queremos analizar si este aumento en el tiempo de ejecución está justificado por una mejora en la calidad de la solución obtenida. Es decir, necesitamos evaluar si el tiempo extra invertido se traduce en una solución significativamente mejor, de forma que

el incremento en la duración del proceso esté justificado por el valor añadido en términos de calidad de la solución.



En el gráfico se observa que, a medida que aumenta el valor de la proporción, la solución obtenida mejora. La reducción del coste observada puede considerarse significativa. Esto se debe a que al contar con un valor de proporción más elevado, se amplía el abanico de ofertas y configuraciones de paquetes disponibles, lo que permite encontrar una solución más óptima. En nuestro caso, donde el objetivo es minimizar el coste, vale la pena aceptar un mayor tiempo de ejecución, ya que el beneficio en la calidad de la solución compensa el incremento en el tiempo de ejecución.

	3962	2781	2037	7852	5759
1,2	685.663	679.730	689.555	646.443	621.028
1,4	661.115	665.932	676.631	654.6805	605.035
1,6	645.263	649.171	667.607	650.2745	595.656
1,8	642.047	637.871	670.441	605.5035	603.330
2	649.908	629.352	671.552	603.493	589.009
2,2	616.000	618.118	669.805	599.163	583.568
2,4	618.946	619.583	654.099	602.665	584.373
2,6	608.949	616.553	653.693	593.931	570.444
2,8	604.145	609.293	641.018	571.606	562.261
3	609.576	615.164	644.760	574.946	570.186

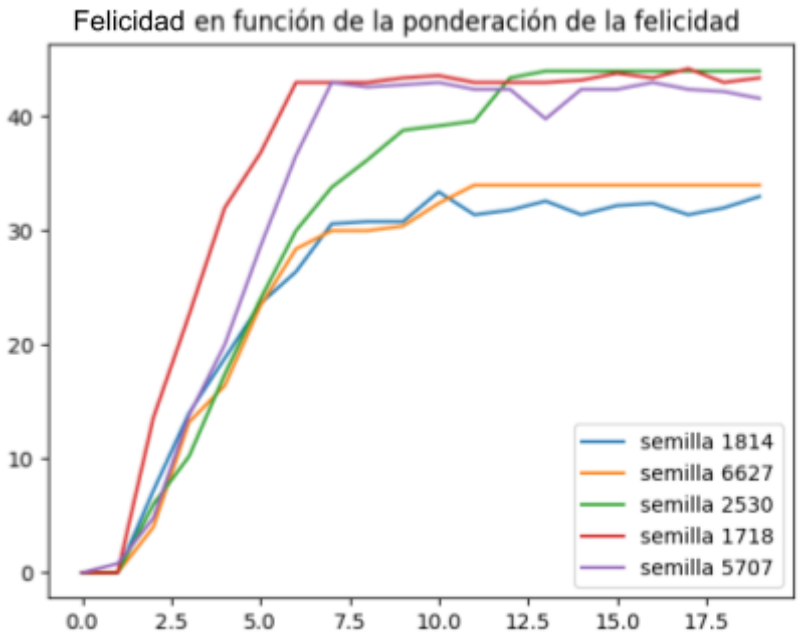
6.6. Estudio de la felicidad a través de la ponderación de la función heurística

El objetivo de este experimento es ver cómo cambia la calidad de la solución final en función de la ponderación que se dé en la función heurística a los dos criterios a optimizar. Debido a la naturaleza del problema no se pueden optimizar ambos de la misma manera ya que entre ellos se encuentra una relación de contrariedad. Por ejemplo, si nos planteamos aumentar la felicidad esto comportará un aumento del coste, no optimizando así el coste. Por ello en este experimento queremos estimar cómo varía el coste y el tiempo de ejecución en función de la ponderación que se realice en la función heurística, en nuestro caso el valor que le demos al parámetro k.

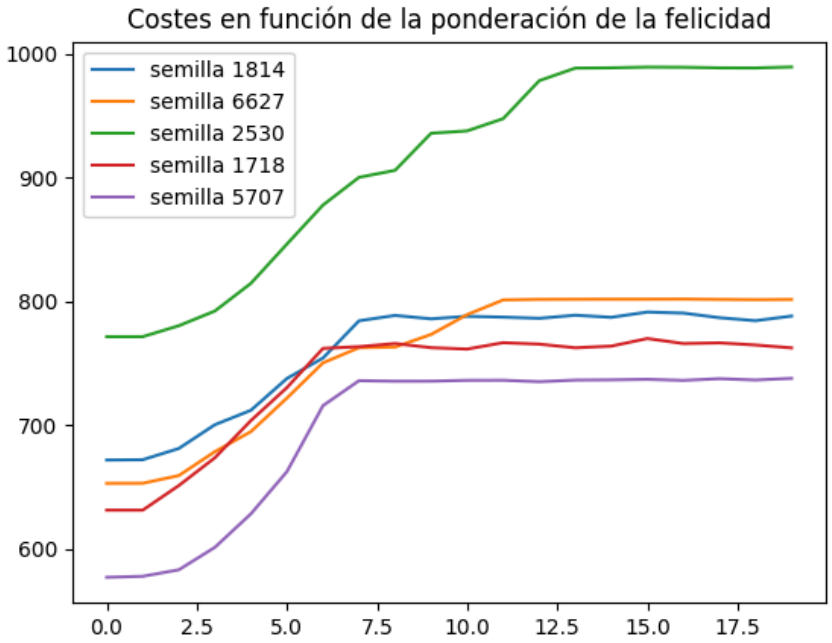
Observación	No podemos optimizar ambos criterios de la función heurística con el mismo peso.
Planteamiento	Le damos un rango de valores al paràmetro k para variar la relevancia que se da a la "felicidad" y analizamos el valor del coste y tiempo de ejecución.
Método	- Usar Hill Climbing - Crear 5 semillas de forma aleatoria - Con cada semilla realizar 10 ejecuciones para obtener mejores resultados - Observar tendéncias en el valor del coste y el tiempo de ejecucion

6.6.1 Resultados

Resultados del coste

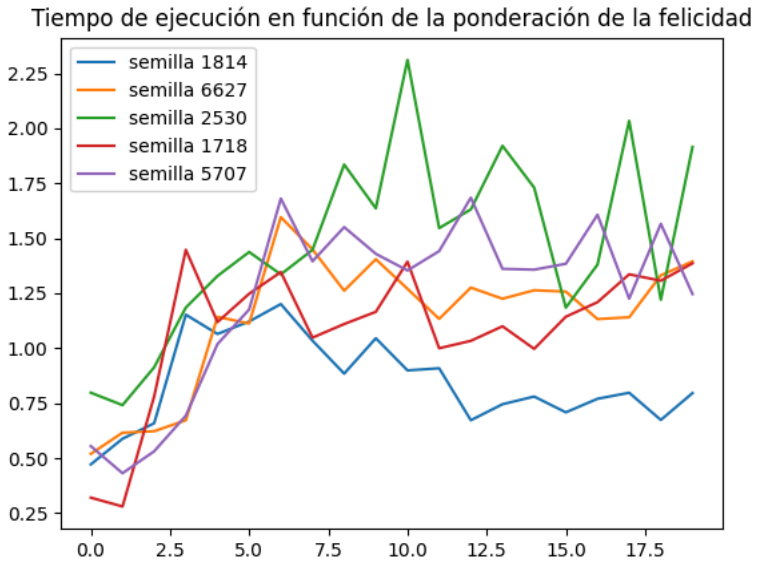


En este gráfico podemos ver cómo a medida que aumenta el valor del parámetro k aumenta también el valor de la felicidad. Llega un punto en el que su valor no puede ser mayor ya que obtiene el valor máximo de felicidad dado el problema, es decir, que respetando las restricciones y maximizando todo lo posible la felicidad se obtiene la mejor asignación de paquetes a ofertas posible.



Por otro lado, este gráfico es de la misma ejecución pero muestra el valor del coste. A medida que aumenta el parámetro k y en consecuencia la felicidad el coste también. En cada ejecución con un k mayor obtenemos peores valores para el coste.

Resultados del tiempo de ejecución



Este gráfico muestra el patrón que sigue el tiempo de ejecución para la misma ejecución realizada con los gráficos anteriores. Vemos que en lo que respecta al tiempo de ejecución no hay ningún patrón apreciable. Lo único que podemos comentar es que al inicio, cuando no tenemos en cuenta la felicidad, el tiempo de ejecución es muy bajo debido a la solución inicial que usamos, por como está planteada, podríamos considerar que minimiza el coste.

6.6.2 Conclusiones

En este experimento se puede estimar como varían los costes de transporte y almacenamiento y el tiempo de ejecución para hallar la solución con el Hill Climbing cambiando la ponderación que se da a este concepto en la función heurística. Y como hemos anticipado a un mayor valor de k, se da más peso a la felicidad y por lo tanto se obtienen peores valores de coste. Además hemos visto que la ponderación del heurístico no tiene ningún impacto en el tiempo de ejecución del problema.

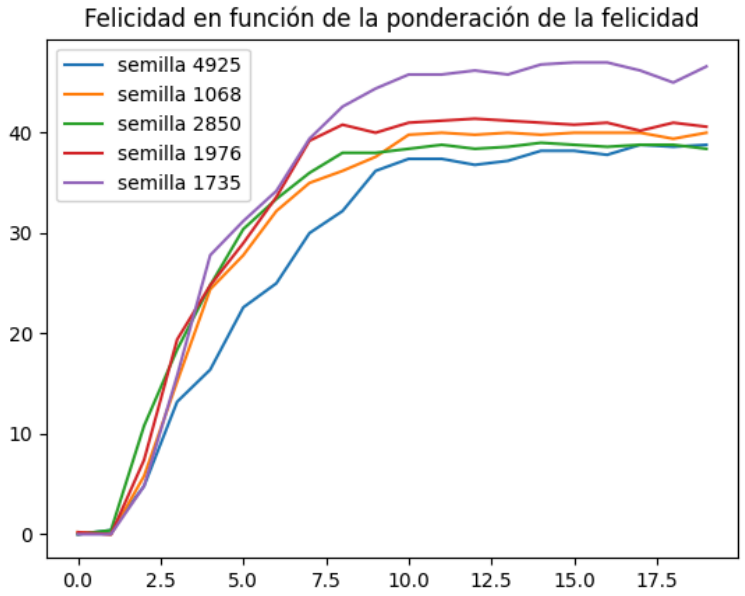
6.7. Comparación con Simulated Annealing

El objetivo de este experimento es detectar qué diferencias se pueden apreciar entre el uso del algoritmo Hill Climbing como en el apartado anterior y el uso de Simulated Annealing. Para ello seguimos exactamente el mismo procedimiento que en el experimento anterior pero usando el algoritmo Simulated Annealing.

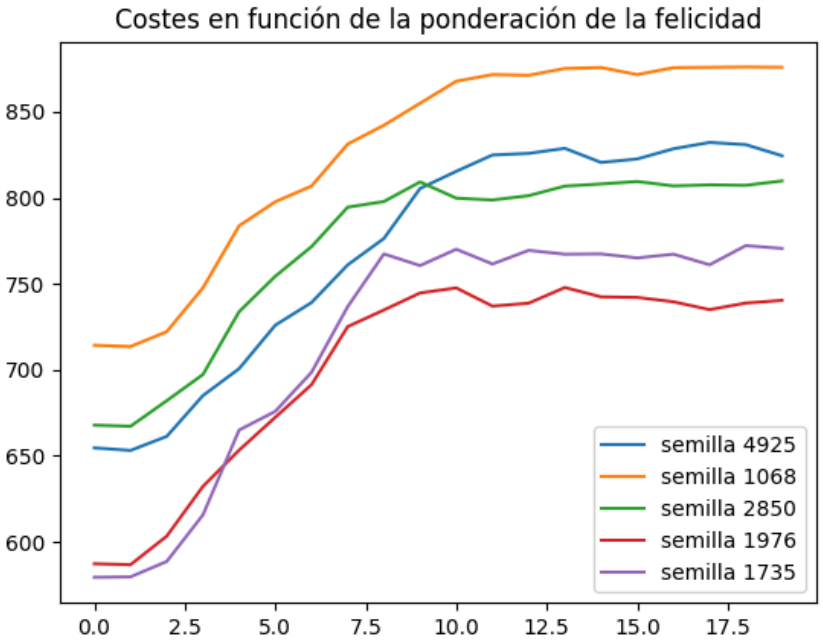
A priori no parece haber ninguna razón para suponer que los resultados serán notablemente distintos. La única diferencia que hemos encontrado anteriormente cuando hemos usado Simulated Annealing es que ha podido llegar a mejores resultados, por lo tanto es posible que el valor del coste no sea tan alto como en el anterior experimento y la felicidad llegue a valores todavía mejores.

6.7.1 Resultados

Resultados del coste

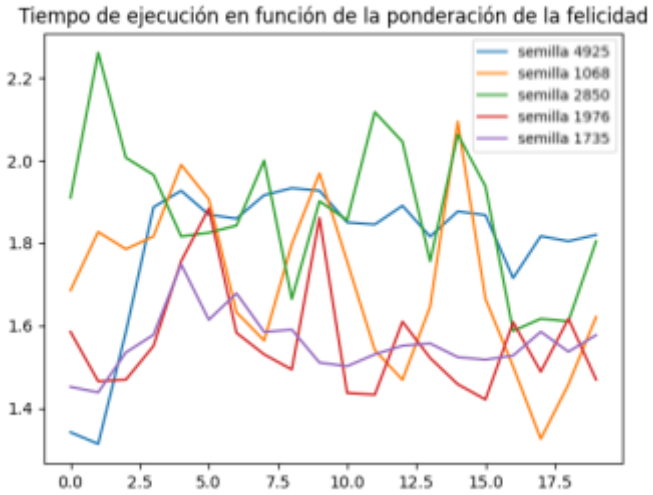


En este gráfico podemos observar prácticamente lo mismo que en el apartado anterior. La único que podríamos considerar es que dependiendo de las semillas puede obtener valores mejores.



Al igual que en el experimento anterior el valor del coste aumenta junto con el valor de k. Si comparamos el valor máximo del coste que obtenemos usando Simulated Annealing que no llega a 900 con el valor cuando usamos Hill Climbing que es casi 1000 vemos que hay una diferencia considerable.

Resultados del tiempo



De nuevo los resultados que obtenemos són prácticamente idénticos al apartado anterior. El valor de la ponderación no tiene ningún tipo de impacto en el tiempo de ejecución del programa.

6.8. Variación del precio fijo de almacenamiento

En caso de variar el coste de almacenamiento a la baja pasaría a ser más rentable dejar los paquetes en el día asignado, lo que haría que mejorar la felicidad implique un mayor incremento en el coste. Por el contrario, incrementar el coste de almacenamiento diario causaría que fuese más rentable adelantar los paquetes, aún a pesar del aumento en el coste de envío. Cuanto más se aumentase el almacenamiento, menor relevancia tendría este incremento en el coste de envío. Esto llevaría a que reducir el coste iría de la mano con reducir la felicidad, por lo que minimizando el coste ya se estaría maximizando la felicidad, o a la inversa.

7. Conclusiones generales

A partir de los experimentos realizados y los resultados obtenidos, hemos podido determinar los operadores y parámetros que nos permiten mejorar significativamente la eficiencia y optimización de la logística y de los parámetros coste y felicidad para nuestra empresa ficticia.

En primer lugar, los experimentos realizados han demostrado que el operador SwapOferta, aunque incrementa el tiempo de ejecución y el número de pasos, mejora la calidad de la solución final, haciéndola especialmente valiosa cuando esta es nuestra prioridad.

También observamos que el uso del estado inicial con componente aleatoria nos permite explorar un espacio de soluciones más amplio, encontrando así soluciones de menor coste, aunque incrementa el tiempo de ejecución y el número de pasos.

Por otro lado, a partir del análisis de diferentes combinaciones de los parámetros del Simulated Annealing confirmamos que una configuración con un valor de k bajo y una λ alta proporciona mejores resultados, equilibrando entre la exploración del espacio de soluciones y la optimización hacia una solución de bajo coste.

En los experimentos también evaluamos cómo el tiempo de ejecución del algoritmo aumenta en relación con el número de paquetes y la proporción de peso transportable en las ofertas. Confirmamos que el aumento de estos valores incrementan el tiempo de ejecución, ya que amplían el estado de soluciones.

En cuanto al coste en función de la cantidad de ofertas disponibles, se observó que una mayor proporción de peso transportable permite obtener soluciones con menores costes, ya que permite más flexibilidad para asignar paquetes y optimizar las ofertas y sus costes.

Por otro lado, estudiamos la felicidad en función de su ponderación en la heurística. Observamos que a medida que se incrementa el peso de la felicidad, el coste tiende a aumentar, mostrando la relación inversa entre estos dos objetivos.

Además, comparamos los algoritmos Hill Climbing y Simulated Annealing en términos de eficiencia y calidad de la solución. Simulated Annealing mostró mejores resultados en cuanto a la minimización del coste y la maximización de la felicidad, ya que su capacidad para aceptar soluciones subóptimas permite evitar óptimos locales y explorar soluciones de mayor calidad.

Finalmente, también experimentamos sobre el impacto del precio fijo de almacenamiento y observamos que al reducir el coste de almacenamiento, se vuelve más rentable almacenar los paquetes, lo que disminuye la felicidad. En cambio, un coste de almacenamiento más alto fomenta el adelanto de los envíos, mejorando la satisfacción del cliente a costa de un aumento en el coste de transporte.

Estos experimentos nos permiten conocer las variables que afectan a la optimización logística y las diferentes estrategias que nos ayudan a mejorar la eficiencia, reduciendo costes y aumentando la satisfacción del cliente en una empresa de distribución como Ázamon.